

CSE 4208: Computer Graphics Laboratory

3D Interactive Apartment Environment

Submitted By

Sumaiya Khan

Roll: 2007031

Submitted To

Dr. Sk. Md. Masudul Ahsan

Professor

Department of Computer Science
and Engineering

Khulna University of Engineering
& Technology

Md Tajmilur Rahman

Lecturer

Department of Computer Science
and Engineering

Khulna University of Engineering
& Technology



**Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh**

Contents

1	Abstract	3
2	Introduction	3
3	Objectives	4
4	Project Proposal	4
4.1	Overview	5
4.2	Proposed Features	5
4.3	Dynamic Interactive Objects	6
4.4	Objects with Complex Shapes	7
5	Software Design	7
5.1	High-Level Data Flow	7
5.2	Source File Organisation	8
6	Rendering Methodology	9
6.1	Pipeline Overview	9
6.2	Geometry Primitives	10
6.3	Custom Projection Matrix	10
6.4	Phong Illumination	10
6.5	Texture Mapping Modes	11
7	Scene Composition	13
7.1	Texture Assets	13
7.2	Ground Floor Rooms	13
7.2.1	TV Room	13
7.2.2	Dining Room	14
7.2.3	Kitchen	14
7.2.4	Ground Floor Bedroom	14
7.3	Stairs, Lift, and Vertical Circulation	14
7.4	Top Floor	15
7.4.1	Bedroom	15
7.4.2	Bathroom	15
7.4.3	Balcony	15
7.5	Outdoor Scene	15
7.5.1	Boundary Wall and Gate	15
7.5.2	Fountain	16
7.5.3	Trees and Vegetation	16

7.5.4	Car and Roads	16
7.5.5	Clouds and Birds	16
8	Bezier Surface Modelling	16
8.1	Mathematical Background	16
8.2	Objects Created	17
9	Ruled Surface (Terracotta Flower Pot)	17
9.1	Mathematical Background	18
9.2	Implementation	18
10	Fractal Tree Generation	18
11	Animation and Interaction	20
12	Camera System and Viewports	20
12.1	6-DOF Camera	20
12.2	4-Split Viewport	21
13	User Controls Reference	21
14	Results and Discussion	22
14.1	Visual Output	22
14.2	Screenshots	23
14.3	Performance	24
15	Conclusion	24
16	Future Work	25

1. Abstract

This report documents the design and development of a real-time 3D apartment environment built with C++ and OpenGL 3.3 Core Profile. The application renders a fully furnished multi-room indoor space featuring a TV room with sofa and bookshelf, a dining room with tables and chairs, a kitchen with full appliances, a ground-floor bedroom, and a top-floor bedroom with an attached bathroom. The outdoor scene includes a grand entrance gate, fountain, fractal trees, animated birds, drifting clouds, and an auto-driving car.

Key graphics techniques demonstrated include per-fragment Phong illumination with three independent light types, switchable texture blending modes (no texture, simple, vertex-blended, fragment-blended), Bézier surface-of-revolution modelling for decorative vases and pots, recursive fractal tree generation, a ruled surface flower pot, skybox rendering, and multi-viewport display. Interactive features include a ceiling fan, opening doors, a moving lift with automated sliding doors, an animated fountain with parabolic water arcs, a tap with water stream animation, an AC unit with louver animation, a table lamp toggle, a TV slideshow, a fridge door, and an auto/manual driving car. On a mid-range system the application sustains 60+ frames per second at 1280×720 resolution.

2. Introduction

Constructing an interactive three-dimensional environment from first principles is one of the most effective ways to internalise the theory taught in a computer graphics course. It forces the developer to confront practical challenges—coordinate system conventions, lighting mathematics, texture memory management, and animation timing—that textbook exercises alone cannot convey.

The goal of this project is to create a virtual apartment complex that looks and feels believable. The scope covers both interior and exterior: a multi-room ground floor, a top-floor bedroom and bathroom connected by a lift, an outdoor yard with trees, a fountain, a boundary wall, and a road network with a moving car. Rather than modelling a single room, the project demonstrates a complete living environment where every space receives attention to detail.

The technology stack consists of GLFW for window creation and keyboard events, GLAD for binding OpenGL 3.3 Core functions, GLM for matrix and vector algebra,

and `stb_image` for decoding JPEG/PNG texture files. A custom projection matrix (`myProjection`) replaces `glm::perspective` to reinforce understanding of the underlying mathematics.

3. Objectives

1. Build the complete OpenGL 3.3 programmable pipeline with custom GLSL vertex and fragment shaders.
2. Hand-code a custom perspective projection matrix to demonstrate the underlying linear algebra.
3. Provide four user-switchable texture modes—no texture, simple texture, vertex-blended, and fragment-blended—to illustrate different mapping strategies.
4. Set up a composite lighting environment with one directional source, two attenuated point sources, and one spotlight, each with independently toggleable ambient, diffuse, and specular components.
5. Model a realistic multi-room apartment: TV room, dining room, kitchen, two bedrooms, a bathroom, a lift, stairs, balcony, and outdoor spaces.
6. Generate smooth surfaces of revolution from Bézier control profiles for decorative vases and pots.
7. Generate a terracotta flower pot using a ruled surface built from spline curves.
8. Produce natural-looking trees via recursive fractal branching.
9. Animate doors, ceiling fans, lift, fountain, birds, clouds, car, tap water, AC louvers, and a table lamp with frame-rate-independent interpolation.
10. Render a realistic skybox cubemap surrounding the entire scene.
11. Offer free-roam camera with 6 degrees of freedom plus a four-quadrant split-screen viewport.

4. Project Proposal

4.1. Overview

This project develops an interactive 3D apartment environment using Modern OpenGL 3.3. The apartment represents a realistic multi-room indoor and outdoor space. The objective is to demonstrate core computer graphics concepts—3D modelling, transformations, lighting, texture mapping, Bézier surfaces, fractal generation, and camera navigation—in a comprehensive, real-time rendered scene.

4.2. Proposed Features

- Complete 3D apartment: TV room, dining room, kitchen, two bedrooms, bathroom, lift, stairs, balcony.
- Outdoor environment: boundary wall, grand entrance gate, fountain, fractal trees, lamp posts, bollard cones, driveway, road network with markings.
- Skybox cubemap rendering for a realistic sky environment.
- Interactive lighting: directional, two point lights, and spotlight—each independently toggleable.
- Four texture blending modes cycled at runtime via keyboard.
- Ceiling fan, fridge door, all room doors, and lift with automated sliding doors.
- Animated fountain with parabolic water arcs.
- Animated bathroom tap with water stream and splash effects.
- AC unit with panel open/close and oscillating louver blades.
- Table lamp toggle with emissive warm glow.
- TV screen slideshow cycling three images every 4 seconds.
- Animated birds (5 birds with flapping wings, orbiting the scene).
- Animated drifting clouds (5 cloud clusters with parallax drift).
- Auto-driving car around a rectangular road circuit; switchable to manual arrow-key control.
- Bezier surface of revolution: decorative vase, arch, dome, pillar, railing.
- Ruled surface: terracotta flower pot with wavy sine-spline top rim.

- Fractal tree: recursive 5-fork branching, depth-4, placed in pots and outdoor.
- 4-split viewport: isometric, top-down, front, and interior views simultaneously.

4.3. Dynamic Interactive Objects

Table 1: Interactive objects and their behaviours

Object	Key	Behaviour
Camera	W/S/A/D/Q/E + I/K/J/H/U/O	6-DOF free roam with pitch/yaw
Ceiling Fan	G	Toggle rotation (continuous Y-axis)
Doors	M/N	Open (90° swing) / Close
Fridge Door	X/P	Toggle open (108° swing with co
Lift	L	Toggle floor (GF↔1F) with auto
Bathroom Tap	4	Toggle water stream, splash, and
AC Unit	8	Toggle panel open and louver os
Table Lamp	0	Toggle warm emissive glow
Car	V	Toggle auto-drive / manual arro
Directional Light	1	Toggle sun
Point Lights	2	Toggle both point lights
Spotlight	3	Toggle camera spotlight
Ambient/Diffuse/Specular	5/6/7	Toggle lighting components
Texture Mode	T	Cycle: no texture → simple → v
Split Viewport	9	Toggle 4-quadrant split screen

4.4. Objects with Complex Shapes

Table 2: Complex object construction from primitives

Object	Construction
Bookshelf	Back panel + side panels + horizontal shelves + 60 upright books with varied colours and heights
Sofa	Seat base + backrest + two armrests + two cushions
Bed (top floor)	Frame with legs, side rails, slats, headboard with decorative studs, mattress, duvet, sheet
Dining Chairs	Seat + 4 legs + backrest slab; 6 chairs arranged around table
Lift Car	Floor, ceiling, back wall, side walls + interior button panel
Fountain	4-sided marble basin + central pedestal + animated water spheres
Car	Layered body + cabin + hood + trunk + windows + lights + 4 wheels with rims and spokes
AC Unit	Body shell + side/top panels + LED indicator + animated front panel + 5 louver blades
Fractal Tree	Cylinder trunk + recursive 5-fork branches (depth 4) + cone leaf tufts at tips

5. Software Design

5.1. High-Level Data Flow

The application initialises GLFW, loads GLAD, compiles three shader pairs (Phong, textured Phong, light-cube emissive), loads 22 JPEG/PNG textures and a 6-face skybox cubemap, constructs all geometry (cube VAO, textured-cube VAO, Bezier revolution surfaces, ruled surface), then enters the render loop.

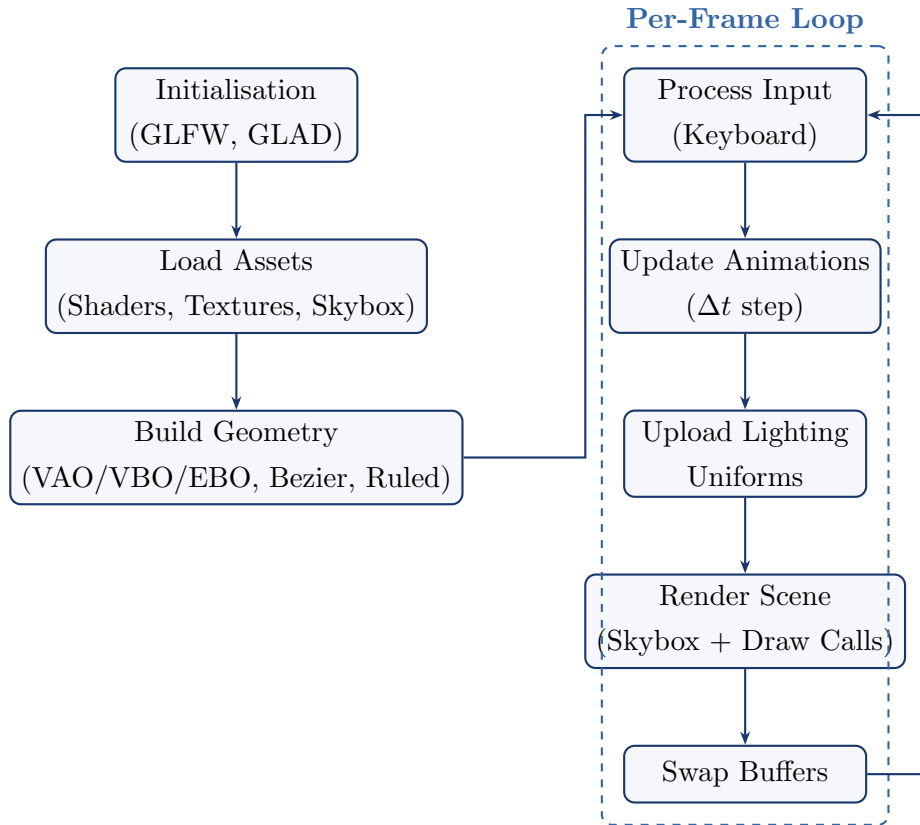


Figure 1: Application data-flow from start-up through the per-frame loop

5.2. Source File Organisation

The codebase is split into a main translation unit and several header-only modules. This layout keeps the project simple (single `.cpp` compile) while still separating concerns clearly.

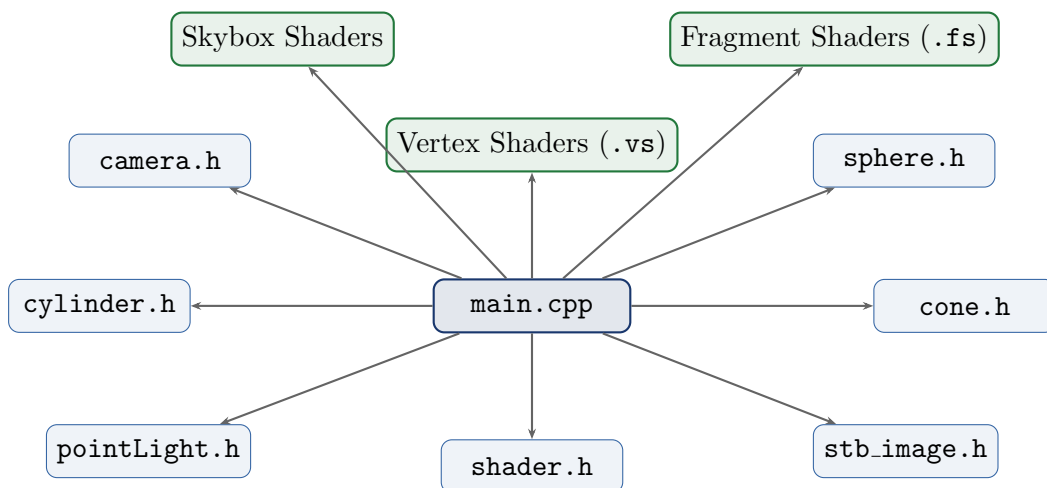


Figure 2: Source file dependency graph

Table 3: Responsibilities of each source module

Module	Responsibility
<code>main.cpp</code>	Application entry point; all scene draw functions; animation state; camera logic; keyboard handler; lift, AC, tap, lamp, fan, car logic; viewport switching
<code>camera.h</code>	6-DOF Euler-angle camera with pitch, yaw, roll and keyboard movement
<code>shader.h</code>	Shader compilation and linking utility
<code>sphere.h</code>	UV-sphere with texture support (<code>drawSphereTexture</code>)
<code>cylinder.h</code>	Parametric cylinder mesh
<code>cone.h</code>	Cone mesh
<code>pointLight.h</code>	Point light setup and toggle
<code>stb_image.h</code>	JPEG/PNG texture loading with 1×1 white fallback
Vertex Shaders	MVP transform; normal-matrix; UV pass-through
Fragment Shaders	Phong from directional + 2 point + spot; 4 texture modes; emissive toggle
Skybox Shaders	Cubemap sampling for surrounding sky

6. Rendering Methodology

6.1. Pipeline Overview

Every frame clears buffers, renders the skybox first (depth function set to `GL_LEQUAL`), then switches to `GL_LESS` for the main scene. The main scene activates the appropriate shader (Phong or textured Phong), uploads global uniforms, and iterates all scene elements. Transparent window glass is rendered last with blending enabled and depth writes disabled.

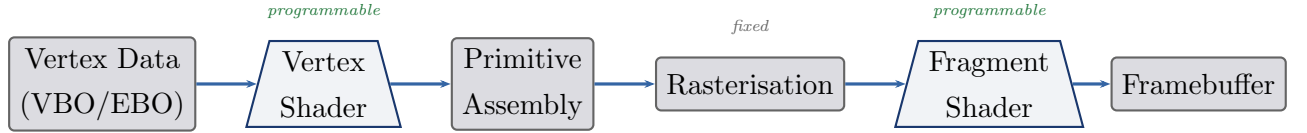


Figure 3: Simplified OpenGL 3.3 pipeline stages (programmable stages shaded)

6.2. Geometry Primitives

Table 4: Primitive mesh specifications

Mesh	Triangles	Usage
Cube (plain)	12	Walls, floors, furniture, fan blades, car body
Cube (textured)	12	Textured surfaces (floor, wall, door, sofa, stairs, lift ...)
Sphere	Variable	Tree canopy, fountain finial, water drops, birds, clouds, flowers
Cylinder	Variable	Flower stems, fractal tree trunk
Cone	Variable	Fractal leaf tufts, bollard path markers
Bezier Revolution	Variable	Vase, arch, dome, pillar, railing
Ruled Surface	Variable	Terracotta flower pot

6.3. Custom Projection Matrix

The standard `glm::perspective` is replaced by a hand-written function `myProjection` that builds the 4×4 symmetric perspective matrix from the half-FOV tangent:

$$\mathbf{P} = \begin{pmatrix} \frac{1}{at} & 0 & 0 & 0 \\ 0 & \frac{1}{t} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -1 \\ 0 & 0 & -\frac{2fn}{f-n} & 0 \end{pmatrix}, \quad t = \tan\left(\frac{\text{FOV}}{2}\right), \quad a = \text{aspect ratio} \quad (1)$$

6.4. Phong Illumination

The fragment shader accumulates light contributions from three categories.

Directional light. Simulates sunlight from direction $(-0.5, -1.0, -0.3)$ with warm white colour. Toggleable via key **1**.

Point lights. Two attenuated sources: a warm red-tinted light at the TV room and a cool blue-tinted light at the dining room. Each is attenuated by:

$$\alpha(d) = \frac{1}{1 + 0.09d + 0.032d^2} \quad (2)$$

Toggleable via key **2**.

Spotlight. A camera-mounted flashlight-style cone with inner cutoff 18° and outer cutoff 26° , with purple-tinted diffuse. Toggleable via key **3**.

The per-source contribution follows:

$$L_i = [k_a + (\hat{\mathbf{n}} \cdot \hat{\mathbf{l}})^+ + k_s (\hat{\mathbf{r}} \cdot \hat{\mathbf{v}})^{32}] \mathbf{C}_i \alpha_i \quad (3)$$

Ambient, diffuse, and specular terms can be toggled individually via keys **5/6/7**.



(a) Spotlight on



(b) Point lights with specular effect



(c) All lights on



(d) Directional and diffuse light on

Figure 4: Lighting system demonstrations

6.5. Texture Mapping Modes

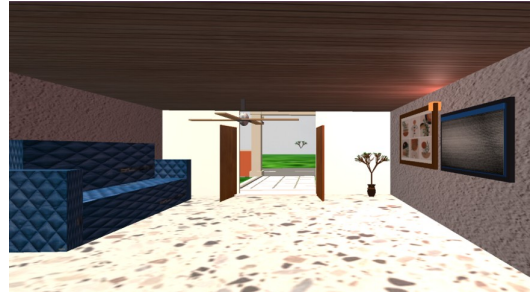
Four modes, cycled by pressing **T**:

Table 5: Texture blending strategies

ID	Name	Base Colour Equation
0	No Texture	$\mathbf{c} = \text{vertexColor}$
1	Simple Texture	$\mathbf{c} = \text{texel}$ (fallback to vertex colour if texel is black)
2	Vertex-Blended	$\mathbf{c} = \text{texel} \times \text{vertexColor}$
3	Fragment-Blended	$\mathbf{c} = \text{mix}(\text{texel}, \text{vertexColor}, 0.5)$



(a) No texture (solid colour)



(b) Simple texture



(c) Vertex-blended texture



(d) Fragment-blended texture

Figure 5: Four texture blending modes applied to the same scene geometry

```

1  vec3 baseColor;
2  if (textureMode == 0)
3      baseColor = objectColor;
4  else if (textureMode == 1)
5      baseColor = texColor;
6  else if (textureMode == 2)
7      baseColor = texColor * objectColor;
8  else
9      baseColor = mix(texColor, objectColor, 0.5);

```

Listing 1: Texture mode logic in the fragment shader

7. Scene Composition

7.1. Texture Assets

Table 6: Loaded textures and their roles

Filename	Applied to
floor.jpg	Indoor floor surfaces
wood.jpg	Exterior walls, kitchen cabinets and countertops
mosaic.jpg	TV room walls and bathroom accent walls
ceiling.jpg	Indoor ceiling slabs
grass.jpg	Outdoor ground terrain
door.jpg	All room doors
roof.jpg	Exterior roof surfaces
stairs.jpg	Staircase treads and risers
sofa.jpg	Sofa body, cushions and arms
tree_leaves.jpg	Outdoor tree canopy spheres
lift.jpg	Lift car walls, floor, ceiling, and doors
books.jpg	Bookshelf book spine faces
tv.jpg / tv2.jpg / tv3.jpg	TV screen slideshow (cycles every 4 s)
portrait1.jpg	Wall portrait in TV room
portrait2.jpg	Wall portrait in dining room and bedroom
cabinet.jpg	Kitchen upper cabinet surface
decor_sphere.jpg	Decorative globe lamp in TV room
bollard.jpg	Outdoor traffic bollard cones
skybox/right.png ...back.png	6-face cubemap skybox

7.2. Ground Floor Rooms

7.2.1. TV Room

The TV room occupies $z = 0$ to 10, with mosaic-textured walls on three sides. A large textured sofa (5 units wide) faces the south wall where a 3.2×2.0 unit TV displays a slideshow of three images cycling every 4 seconds. A bookshelf unit against the east partition wall holds 60 upright books in 5 rows with 10 colours. A

decorative sphere lamp hangs from the ceiling, and a fractal-tree pot sits in the corner. A ceiling fan above the room is toggled with key G.

7.2.2. Dining Room

The dining room occupies $z = 0$ to 10 on the west side. A 7×3 dining table seats 6 chairs arranged on all four sides. A Bézier vase with a flower bouquet (3 flowers with 8-petal symmetric geometry) sits at the table centre. A fruit bowl with red apple, green apple, orange, and banana is placed to the right. Two fractal-tree pots occupy the corners. A portrait frame is mounted on the back wall ($z = 10$).

7.2.3. Kitchen

The kitchen runs along $x = 8$ to 10, $z = 0$ to 10. Wood-textured lower cabinets carry a cabinet-textured countertop. Upper wall cabinets hold mugs and food boxes. Counter objects include a cutting board with knife, stacked plates, a frying pan with handle, a large cooking pot, and a fruit bowl. The refrigerator ($5 \times 1.7 \times 1.5$ units) stands beside the counter with a fully animated bottom door (opens 108° on key X/P) and visible groceries inside.

7.2.4. Ground Floor Bedroom

The bedroom ($x = 4$ to 10, $z = -20$ to -12) contains a large bed with base and mattress, bedside steps, and a dressing table with a mirror, makeup items, jewellery tray, and a small lamp. A portrait is mounted on the right side wall. An AC unit is mounted high on the back wall, toggled with key 8: the front panel tilts open and 5 louver blades oscillate sinusoidally.

7.3. Stairs, Lift, and Vertical Circulation

Two staircases connect the floors: a 10-step staircase going from ground to first floor along $x = 8$, and a second 10-step run at $x = 6$ going from first to second floor. Both use textured treads and risers with a solid support block under each step to avoid a floating appearance.

The lift shaft ($x = 6$ to 10, $z = -4$ to 0) contains the lift car which travels between $y = -1$ (ground floor) and $y = 4$ (first floor) when key L is pressed. Sliding doors open automatically when the car arrives. An interior button panel shows floor buttons with colour coding, and landing call panels are placed outside on both floors.

7.4. Top Floor

The top floor ($x = -4$ to 3 , $z = -15$ to 5 , $y = 4$) is a unified bedroom and bathroom space divided by a partial wall at $z = -5$.

7.4.1. Bedroom

A double bed with headboard, mattress, duvet, and sheet is placed against the back wall. A nightstand holds a table lamp (key **0**) with an emissive warm-glow shade and bright sphere bulb. A study table with monitor/mirror, drawer cabinet, and desk lamp is against the right wall, paired with a modern metal-framed office chair. A full wardrobe with two door panels and handles is flush against the left wall. A layered rug and a corner plant complete the room. A second ceiling fan (keyed to G) hangs from the bedroom centre.

7.4.2. Bathroom

The bathroom features a large freestanding rectangular bathtub (pure white, $3.6 \times 1.6 \times 0.8$ units) with a floor-standing gooseneck faucet. When key **4** is pressed, an animated water stream falls from the spout with parabolic drop trajectories, splash droplets, and ripple rings on the water surface. The toilet has a full cistern, lid, seat, flush button, and water supply pipe. The washbasin uses a Bézier-vase pedestal with a sphere-based bowl, gooseneck faucet, mirror, and soap dispenser. A towel rack and medicine cabinet are on the right wall.

7.4.3. Balcony

The balcony houses a wooden porch swing (dolna) with an A-frame structure, rope hangers, and a cushioned seat, and a rocking chair with rockers, slat backrest, and armrests. Both face the entrance gate direction. A terracotta ruled-surface flower pot with a green bush sphere sits in the corner.

7.5. Outdoor Scene

7.5.1. Boundary Wall and Gate

A brick-coloured boundary wall encloses the property (x : -20.2 to 20.2 , z : -30.2 to 20.2) with cap stones, corner pillars, and a gap on the left side for the grand entrance gate. The gate features two stone pillars with plinths, shaft, capital, groove lines, and sphere finials. A Bézier arch decorates the gate opening and a gold name plate spans the lintel.

7.5.2. Fountain

A marble-toned fountain with a 4-sided basin, central pedestal, and sphere finial sits in the left yard. The water animation uses 8 radial parabolic streams of small spheres, each following a sine-based trajectory, updated every frame for smooth flowing motion.

7.5.3. Trees and Vegetation

Nine sphere-canopy trees (3 sphere layers per tree with tree-leaves texture) are placed symmetrically in both yards and the back yard. Three fractal trees (5-fork branching, depth 4) are placed near the gate and side walls. Three pot-trees using Bézier vases with pebble soil and fractal trunks are placed inside the dining room corners and TV room corner. Bezier decorative vases flank the building entrance.

7.5.4. Car and Roads

A detailed car (body, cabin, hood, trunk, 4 windowed sides, 4 wheels with tyre walls, rims, hub caps, and cross spokes) auto-drives counter-clockwise around a rectangular road circuit on key **V** or is controlled manually with arrow keys. A second car is parked inside the garage. Roads have white centre-dashed markings and parking bay lines.

7.5.5. Clouds and Birds

Five animated cloud clusters (each built from 5 squashed sphere puffs) drift across the sky with sinusoidal parallax offsets, each at a unique altitude and drift speed. Five animated birds orbit the scene at varying radii and altitudes; each bird has a body box and two wing boxes that flap via a sine function.

8. Bezier Surface Modelling

8.1. Mathematical Background

Given $L + 1$ control points in the radial–height (r, y) plane, the Bézier curve of degree L at parameter t is:

$$\mathbf{B}(t) = \sum_{i=0}^L \binom{L}{i} (1-t)^{L-i} t^i \mathbf{P}_i, \quad t \in [0, 1] \quad (4)$$

To form a closed 3D surface, the profile is revolved 360° about the Y-axis. At

each ring t_j , the curve yields radius r_j and height y_j ; sweeping through azimuthal angle ϕ_k gives:

$$(x, y, z) = (r_j \cos \phi_k, y_j, -r_j \sin \phi_k) \quad (5)$$

8.2. Objects Created

Table 7: Bézier revolution objects and their control point degrees

Object	Degree	Placement
bezVase	8	Dining table centrepiece, dining room pot-trees, entrance flanking, bedroom pot-tree, bathroom basin pedestal
bezArch	4	Grand entrance gate arch decoration
bezDome	3	Decorative dome (available)
bezPillar	4	Architectural pillar (available)
bezRailing	8	Balcony railing spindles (north and south balconies, 40+ spindles)

```

1 static glm::vec2 evalBez(float t, float* cp, int deg) {
2     float x = 0, y = 0;
3     for (int i = 0; i <= deg; i++) {
4         double c = (double)nCr(deg, i)
5             * pow(1.0 - t, deg - i) * pow(t, i);
6         x += (float)(c * cp[i * 2]);
7         y += (float)(c * cp[i * 2 + 1]);
8     }
9     return glm::vec2(x, y);
10 }
```

Listing 2: Bézier point evaluation (C++)

9. Ruled Surface (Terracotta Flower Pot)

9.1. Mathematical Background

A ruled surface is formed by linearly interpolating between two boundary curves. For two profile curves $\mathbf{C}_1(t)$ (top rim) and $\mathbf{C}_2(t)$ (base circle):

$$\mathbf{S}(t, s) = (1 - s) \mathbf{C}_1(t) + s \mathbf{C}_2(t), \quad s \in [0, 1] \quad (6)$$

9.2. Implementation

The top rim profile uses a wavy sine-based spline:

$$r_{\text{top}}(\theta) = 0.6 + 0.08 \sin(8\theta), \quad \theta \in [0, 2\pi] \quad (7)$$

The base profile is a smaller stable circle with $r_{\text{bot}} = 0.35$. The ruled surface connects 41 sample points on each profile, generating a mesh of 40 quad strips (80 triangles). Vertex normals are computed analytically via the cross product of the tangent and the ruling direction, ensuring smooth Phong shading. The pot is placed on the top-floor balcony and carries a green sphere bush.

```
1 float rTop = 0.6f + 0.08f * sinf(angle * 8.0f);  
2 pts1.push_back(glm::vec3(rTop * cosf(angle), 1.2f,  
3                           rTop * sinf(angle)));
```

Listing 3: Top rim spline computation (C++)

10. Fractal Tree Generation

Each tree is grown by recursive branching:

1. A vertical trunk cylinder is drawn from the base.
2. From the trunk tip, 5 forks emerge at pre-defined azimuthal and tilt angles: forward, right-front, right-back, left-front, left-back.
3. Each fork recursively spawns 5 more branches.
4. At each level the branch length shrinks by $0.68\times$ and radius by $0.62\times$.
5. At depth 0 (terminal), a cone leaf tuft is drawn at the tip.
6. Fractal trees are placed: 3 outdoor fractal trees (depth 4), 3 pot-trees indoors (depth 3), and 25 additional scattered neighbourhood trees.



Figure 6: Schematic of the recursive 5-fork branching pattern (depth 4)

```

1 float forks[][3] = {
2     {0.f, 25.f, 0.f},      // forward tilt
3     {72.f, 30.f, 10.f},   // right-front
4     {144.f, 28.f, -8.f},  // right-back
5     {-72.f, 30.f, -10.f}, // left-front
6     {-144.f, 28.f, 8.f}   // left-back
7 };
8 for (int i = 0; i < 5; i++) {
9     glm::mat4 t = glm::rotate(tip, glm::radians(forks[i
10        ] [0]),
11                                glm::vec3(0,1,0));
12     t = glm::rotate(t, glm::radians(forks[i][1]),
13                        glm::vec3(1,0,0));
14     drawFractalBranch(sh, bark, leaf, t, cL, cR, depth-1)
15     ;
16 }

```

Listing 4: Fractal branching (C++ excerpt)

11. Animation and Interaction

All toggled animations use delta-time scaling, ensuring consistent speed regardless of frame rate.

Table 8: Animated elements and their parameters

Element	Key	Motion
Ceiling Fan (ground floor + bedroom)	G	Continuous rotation at $150^\circ/\text{s}$
Room Doors (all)	M/N	$0 \rightarrow 90^\circ$ swing at $80^\circ/\text{s}$
Fridge Door	X/P	$0 \rightarrow 108^\circ$ smooth lerp
Lift Car	L	$\text{GF} \leftrightarrow \text{1F}$ at 2.5 units/s; doors auto-slide
Fountain Water	—	8 radial parabolic streams, continuous
Bathroom Tap Water	4	Parabolic stream drops + splash + ripple rings
AC Panel	8	Front panel tilts 35° ; 5 louvers oscillate $\pm 25^\circ$
Table Lamp	0	Emissive warm shade + light pool toggle
TV Screen	—	3-image slideshow cycling every 4 s
Car (auto)	V	Counter-clockwise road circuit at 8 units/s
Birds (5)	—	Orbit at varying radii/heights; wings flap via sine
Clouds (5 clusters)	—	Drift with sinusoidal parallax; gentle vertical bob
Fountain Sphere	—	Continuous water arc trajectories

```

1 if (openDoor && doorAngle < 90.0f)
2     doorAngle += 80.0f * deltaTime;
3 if (!openDoor && doorAngle > 0.0f)
4     doorAngle -= 80.0f * deltaTime;

```

Listing 5: Frame-rate-independent door animation

12. Camera System and Viewports

12.1. 6-DOF Camera

The camera stores **Position**, **Pitch**, **Yaw**, and **Roll**. Keys W/S/A/D/Q/E translate along the front/right/up axes; I/K pitch up/down; J/H yaw left/right; U/O

roll left/right. Holding LEFT SHIFT multiplies movement speed by 4. Quick-snap presets:

Table 9: Quick-snap camera presets

Key	View
B	TV room interior (facing south wall)
C	Dining room interior (facing back wall)
F / Z	Ground floor bedroom
R	Top-floor elevated overview

12.2. 4-Split Viewport

Key **9** toggles a 4-quadrant split, each rendered independently with `glViewport` and `glScissor`:

- **Top-left:** Isometric perspective from $(25, 20, 25)$.
- **Top-right:** Top-down orthographic from $(0, 30, 0)$ (ambient-only lighting).
- **Bottom-left:** Front exterior view from $(-38, 4, -5)$ (diffuse-only lighting).
- **Bottom-right:** Fixed interior view from $(-8.5, 2.5, 9)$ (directional-only lighting).

Each quadrant also renders the skybox with the corresponding view matrix.

13. User Controls Reference

Table 10: Complete keyboard controls

Key	Action
W / S	Camera forward / backward
A / D	Camera strafe left / right
E / Q	Camera up / down
I / K	Camera pitch up / down
J / H	Camera yaw left / right
U / O	Camera roll left / right
LEFT SHIFT	Hold for $4\times$ speed multiplier

Key	Action
B	Snap to TV room view
C	Snap to dining room view
F / Z	Snap to ground bedroom view
R	Snap to top-floor overview
G	Toggle ceiling fans
M / N	Open / Close all doors
X / P	Toggle fridge door
L	Toggle lift (GF $\leftrightarrow$ 1F)
4	Toggle bathroom tap water
8	Toggle AC unit
0	Toggle table lamp
V	Toggle car auto-drive / manual
Arrow Keys	Manual car steering (when auto-drive off)
T	Cycle texture mode (0 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 0)
1	Toggle directional light
2	Toggle point lights
3	Toggle spotlight
5 / 6 / 7	Toggle ambient / diffuse / specular
9	Toggle 4-split viewport
ESC	Quit

14. Results and Discussion

14.1. Visual Output

The rendered scene faithfully recreates the look and feel of a modern apartment complex. Notable visual outcomes include:

- A fully furnished multi-room interior with consistent Phong-lit surfaces and visible specular highlights on mirrors, glass, and metal fixtures.
- Four texture blending modes producing visually distinct appearances for the same geometry.
- Smooth door, fridge, lift, and AC animations with frame-rate-independent interpolation.
- A continuous skybox surrounding the scene with no visible seams.

- Realistic animated fountain with 8 radial parabolic water streams.
- Bathroom tap producing a falling water column with splash rings and a filling water level in the bathtub.
- 5 animated birds with flapping wings and 5 animated cloud clusters with organic drift.
- Detailed car with layered wheel geometry driving autonomously around the road circuit.
- Bezier vases visible in 5 different locations; balcony railing with 40+ Bezier spindles.
- Terracotta ruled-surface flower pot with a green sphere bush on the balcony.
- Fractal trees both indoors (pots) and outdoors (yard and gate), each with unique branching spread.

14.2. Screenshots

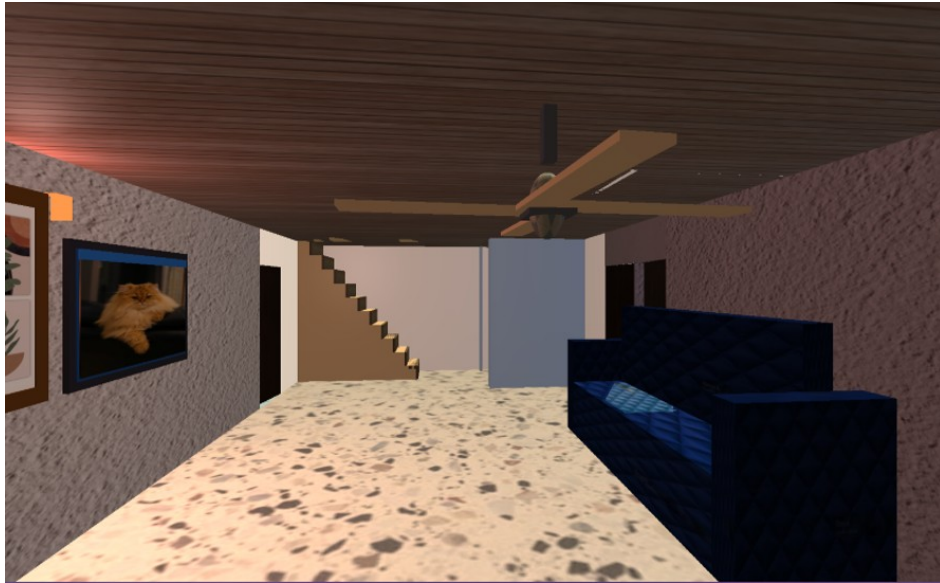


Figure 7: Camera perspective inside the apartment (TV room facing south wall)

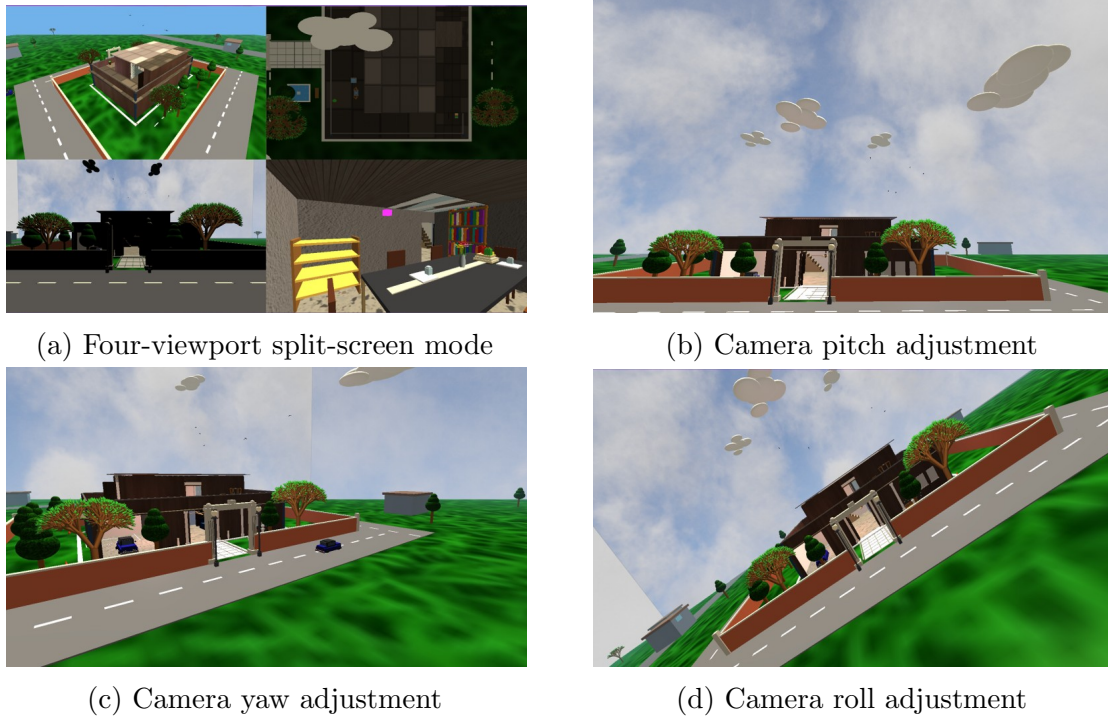


Figure 8: Alternative camera viewing angles demonstrating 6-DOF navigation



(a) Outdoor scene with gate, fountain, and trees

14.3. Performance

On a mid-range GPU the application renders at a steady **60+ FPS** at 1280×720 . The scene issues several thousand draw calls per frame (one per cube instance), remaining within the driver overhead budget for this resolution.

15. Conclusion

This project delivers a complete, interactive 3D apartment environment using OpenGL 3.3 and C++. Every objective listed in Section 3 has been fulfilled:

- Custom vertex and fragment shaders implement Phong lighting with per-component toggles and four texture blending modes.
- A hand-written perspective projection matrix confirms understanding of the linear algebra behind the GPU pipeline.
- A rich interior and exterior scene—TV room, dining room, kitchen, two bedrooms, bathroom, lift, stairs, balcony, fountain, gate, boundary wall, road network—demonstrates effective scene composition.
- Bézier surface-of-revolution and ruled-surface generation extend the project beyond basic cube stacking into procedural geometry modelling.
- A fractal tree with 5-fork recursive branching provides natural-looking organic structures.
- Ten distinct interactive animations (fan, doors, fridge, lift, fountain, tap water, AC, table lamp, TV slideshow, car) provide a polished interactive experience.
- A skybox cubemap and 4-split viewport offer flexible scene exploration.

Working on this project reinforced the importance of consistent coordinate conventions when thousands of object instances share the same world space, and highlighted how procedural techniques (Bézier surfaces, fractal branching, ruled surfaces) dramatically increase visual richness with minimal extra code.

16. Future Work

- **Shadow mapping** for directional and point lights to add depth cues.
- **Physically based rendering** (PBR) for more realistic wood, metal, and glass materials.
- **Normal and bump mapping** on walls and floors to simulate surface roughness.
- **Mouse-driven camera** with cursor look and scroll-wheel zoom.
- **Day/night cycle** by smoothly ramping directional light colour and intensity.
- **Instanced rendering** to reduce draw calls for repeated objects (books, bollards, road markings).

- **Ambient occlusion** for more realistic shadows in corners and crevices.
- **Skeletal animation** for a walking character figure in the apartment.

References

- [1] Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3* (8th ed.). Addison-Wesley.
- [2] Akenine-Möller, T., Haines, E., & Hoffman, N. (2018). *Real-Time Rendering* (4th ed.). A K Peters/CRC Press.
- [3] GLFW Project (2023). *GLFW 3 Reference Documentation*. <https://www.glfw.org/docs/latest/>
- [4] Riccio, C. (2023). *OpenGL Mathematics Library*. <https://glm.g-truc.net/>
- [5] Barrett, S. (2023). *stb_image – single-header image loader*. <https://github.com/nothings/stb>
- [6] de Vries, J. (2020). *LearnOpenGL*. <https://learnopengl.com/>
- [7] Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F. (1996). *Computer Graphics: Principles and Practice* (2nd ed. in C). Addison-Wesley.
- [8] Farin, G. (2002). *Curves and Surfaces for CAGD* (5th ed.). Morgan Kaufmann.